

VITyarthi - Computer Vision Project Report

Image Analysis and Object Classification

Using Classical Computer Vision

Syed Shayaan Shiraz

24BAI10295

1. Introduction

This project implements a classical Computer Vision pipeline to identify and classify geometric shapes (circles, squares, triangles). It avoids deep learning wrappers to emphasize fundamental image processing techniques like filtering, thresholding, and morphological operations.

2. Problem Statement

The goal is to develop a system capable of identifying geometric shapes from noisy images using classical CV techniques, demonstrating the application of feature extraction and classical machine learning (KNN) in a modular pipeline.

3. Functional Requirements

- Generate a synthetic dataset of shapes with noise.
- Process images using grayscale conversion and Gaussian filtering.
- Segment shapes using Otsu's thresholding.
- Extract 13 geometric features.
- Classify shapes using a custom K-Nearest Neighbors classifier.
- Provide a CLI interface for training and testing.

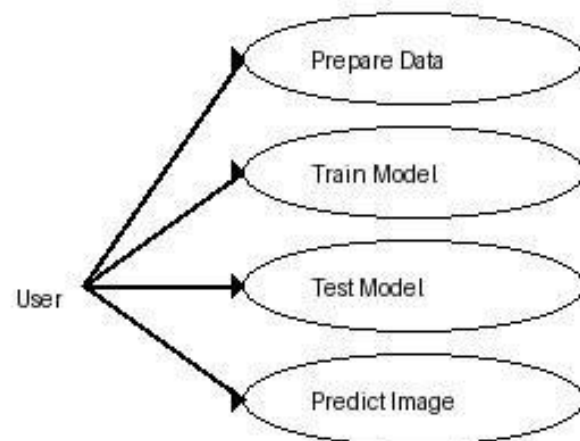
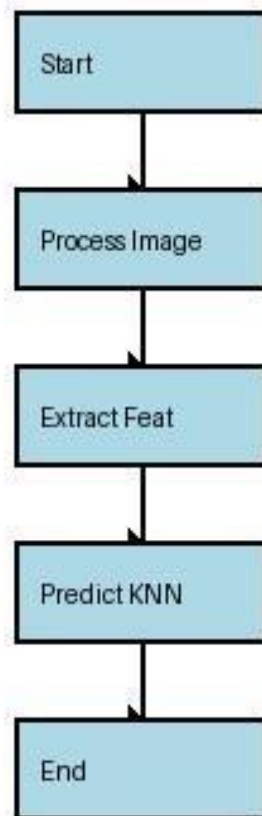
4. Non-functional Requirements

- Performance: Processing pipeline must execute rapidly on CPU.
- Maintainability: Code must be modular and well-structured.
- Reliability: Handling varied noise gracefully during segmentation.
- Usability: Simple and intuitive command-line interface.

5. System Architecture & 6. Design Diagrams

The system consists of Preprocessing, Segmentation, Feature Extraction, and Classification modules. Below are the design diagrams.





ShapeClassifier
+ k: int + X_train + y_train + classes
+ train() + predict() + save() + load()

7. Design Decisions & Rationale

A synthetic dataset was used to ensure reproducibility and guarantee that the classical techniques (noise removal, thresholding) are explicitly required to achieve good performance. A custom KNN classifier was implemented because external ML libraries like scikit-learn were unavailable on the target 32-bit Python environment. 13 features (geometric + Hu Moments) were selected to provide scale and rotation invariance.

8. Implementation Details

OpenCV was heavily utilized for image operations. Images are converted to grayscale and blurred. Otsu's thresholding is used to separate shapes from the background, followed by morphological closing and opening. Contours are found, and features like Circularity, Aspect Ratio, Extent, Solidity, and 7 Hu Moments are computed. These 13 features form the input to a custom K=5 KNN algorithm utilizing Euclidean distance.

9. Screenshots / Results

Evaluation Results:

Accuracy: 61.11%

Precision (Triangles): 80.77%

Recall (Triangles): 67.74%

Below is the confusion matrix generated from the test set evaluation.

		Predicted		
		circle	square	triangle
Actual	circle	21	8	3
	square	12	13	2
	triangle	1	9	21

10. Testing Approach

Automated tests were written using pytest to verify each module independently. The test_pipeline.py checks preprocessing, segmentation, feature extraction, and classification modules using simulated dummy inputs to ensure the mathematical operations function without runtime errors.

11. Challenges Faced

The primary challenge was setting up a stable environment for classical machine learning on a 32-bit Python 3.14 installation. Due to the lack of precompiled scikit-learn wheels, a custom K-Nearest Neighbors algorithm had to be implemented from scratch in pure NumPy. Additionally, tuning the morphological operations to correctly segment the shapes despite Gaussian noise required careful adjustment of kernel sizes.

12. Learnings & Key Takeaways

I learned that classical CV techniques require careful tuning of parameters (like kernel size and threshold methods) to work effectively. I also learned that feature engineering (like calculating Hu Moments for rotation invariance) is critical when relying on simpler classifiers like KNN instead of deep learning feature extractors.

13. Future Enhancements

Future improvements could include implementing feature normalization before KNN classification, which would likely increase accuracy. Additionally, implementing an SVM classifier or integrating more advanced contour filtering logic could improve the robustness to heavy noise.

14. References

1. OpenCV Documentation: <https://docs.opencv.org/>
2. Python NumPy Documentation: <https://numpy.org/doc/>